

Algorithm: RNN (Hyperparameter tuning)**Dataset: Amazon Reviews Dataset (Twitter Sentiment Analysis Dataset)****Link to dataset: [Dataset](#)**

Task	Actions Taken	Rationale	Difficulty Level (1-10)	Time Taken
Data Understanding	Explored the sentiment analysis dataset, analyzed text data distribution, and identified the target variable.	To understand dataset structure and ensure suitability for binary sentiment classification.	3	30 min
Data Preprocessing	Preprocessed text data, removed special characters, and converted text into numerical embeddings using TensorFlow Hub (GNews Swivel-20dim).	Text preprocessing ensures meaningful input representation, improving model performance.	5	40 min
Implemented Initial RNN Model	Built a basic RNN-based sentiment classifier with SimpleRNN layers and a dense output layer.	To establish a baseline model for sequential text classification.	6	45 min
Experimented with Embedding Layers	Tested different pretrained word embeddings, including NNLM and Universal Sentence Encoder.	Different embeddings impact feature representation and overall model effectiveness.	7	40 min

Adjusted RNN Architecture	Modified RNN units, added more layers, tested LSTM/GRU variations, and applied dropout regularization.	To analyze the impact of deeper networks and alternative RNN types.	8	1.5 hours
Optimized Hyperparameters	Tuned learning rates, batch sizes, and epochs; tested different optimizers (Adam, RMSprop, Nadam).	To optimize training stability, convergence speed, and overall accuracy.	7	1 hour
Evaluated Model Performance	Compared accuracy and loss trends across different architectures and hyperparameter settings.	To identify the best-performing configuration for sentiment classification.	6	45 min
Implemented Batch Normalization & Dropout	Added batch normalization and dropout layers to improve generalization and prevent overfitting.	To stabilize training and improve model robustness on unseen data.	7	1 hour
Final Model Selection	Selected the best-performing RNN configuration based on validation accuracy and loss.	To ensure the model generalizes well and avoids overfitting.	5	30 min
Conclusion & Analysis	Summarized findings, documented key insights, and outlined potential future improvements.	To refine the model and prepare for real-world application.	4	30 min

1. Business Understanding:

Objective:

The objective of this work is to develop an optimized Recurrent Neural Network (RNN)-based text classification model for sentiment analysis using the Amazon Reviews dataset. The objective is to explore various architectural and hyperparameter tuning approaches to enhance model performance, generalize better, and minimize overfitting.

To achieve this, the current work experiments with several embedding layers, RNN configurations, dropout, batch normalization, and optimizers in order to ascertain the best combination for the maximum accuracy and minimum validation loss. The derived model should be robust, efficient, and effectively capable of sentiment prediction with excellent generalization on unseen data.

2. Data Understanding:

The dataset consists of tweets with sentiments (negative or positive) and is therefore suitable for binary sentiment classification. It is commonly used in Natural Language Processing (NLP) tasks for model training on sentiment analysis.

Key Features:

tweet: The raw text data (user's tweet).

label: The sentiment label (0 = negative, 1 = positive).

3. Data Preparation:

- Text Cleaning: Removing special characters, URLs, and unwanted symbols.
- Tokenization: Splitting sentences into words.
- Embedding Representation: Translating text into numerical vectors through pretrained embeddings (Swivel, BERT, etc.).
- Train-Test Split: Dividing the dataset into training (80%) and testing (20%) for model assessment.

4. Modeling -Baseline Implementation:

- Embedding Layer: Utilized pretrained word embeddings from TensorFlow Hub (GNews Swivel-20dim) to represent text
- Recurrent Neural Network (RNN): Used SimpleRNN layers for representing sequential data.
- Optimization Algorithm: Used Adam optimizer with initial learning rate set to 0.001.
- Initial Hyperparameters:
 - RNN Units: 64 per layer.
 - Epochs: 10 (configurable).

- Batch Size: 32.
- Loss Function: Binary Cross-Entropy for sentiment classification.
- Activation Function: Sigmoid in the output layer for binary classification.
- Training Strategy: Trained on an 80-20 train-test split with validation accuracy monitoring.
- Performance Metrics: Accuracy and loss calculated at each epoch.
- Baseline Performance: Achieved accuracy of 93.76% on the training set and 93.01% on the validation set.

5. Evaluation:

Experiment	Accuracy	Loss	Validation Accuracy	Validation Loss	Observations
Initial Model	0.9376	0.1743	0.9301	0.1934	Strong baseline performance. The model performs well without any optimization or added layers, achieving high accuracy and low loss.
Change the Embedding Layer	0.9419	0.1698	0.9357	0.1846	Improved performance with a different embedding layer. A slight increase in accuracy and validation accuracy.
Changing RNN Units	0.9373	0.1785	0.9385	0.1802	Slight drop in training accuracy. Increasing units has minor effect on accuracy but slightly improved validation accuracy.
Add More Layers	0.9435	0.1597	0.9368	0.1833	Additional layers improve the model's complexity and slightly enhance training and validation accuracy.
Tune Hyperparameters (Learning rate = 0.001)	0.9457	0.1531	0.9377	0.1811	Hyperparameter tuning with a different learning rate and batch size showed improvement in training accuracy and validation accuracy.
Adding Dropout Layer	0.9523	0.1307	0.9359	0.1928	Adding dropout layers greatly improved training accuracy, suggesting a reduction in overfitting. However, validation accuracy slightly dropped.
Adding BatchNormalization	0.9623	0.1017	0.9334	0.2094	BatchNormalization helped achieve the best training accuracy but resulted in a slight increase in validation loss, indicating potential overfitting.
Using RMSprop Optimizer	0.9484	0.155	0.9366	0.1852	RMSprop optimizer showed stable performance with slightly improved training accuracy. Validation accuracy remained on par with previous models.
Using Nadam Optimizer	0.9478	0.1505	0.9376	0.1821	Nadam performed similarly to RMSprop, with a marginally higher validation accuracy, making it a good alternative optimizer.

The experiments attempted to systematically examine the impact of architectural modifications and hyperparameter tuning on RNN performance in text classification. Various configurations were tested, including embedding layer adjustment, increasing the number of RNN units, and adding additional layers to test their impact on feature learning and sequence learning. Hyperparameter tuning focused on adjusting learning rate, batch size, and epochs to enhance model convergence. Regularization techniques such as dropout and batch normalization were

brought in to study how they would prevent overfitting. Different optimizers such as RMSprop and Nadam were also experimented with to quantify their effect on training stability and convergence speed. The general goal was to determine the best-performing combination of model architecture and optimization methods for robust text classification.

6. Observation:

- i. Initial Model: The first model performed very well, and it was 93.76% accurate and had a validation accuracy of 93.01%. This model did not use any changes like layer change, hyperparameter tuning, or advanced methods, so it is a good starting point.
- ii. Replace the Embedding Layer: Replacing the embedding layer improved performance by a small margin. The model increased by 0.15% accuracy on the validation set, which indicates that changing embedding representations can have a great effect on the model's understanding of the input text.
- iii. Changing RNN Units: Changing the number of SimpleRNN units did not make a significant difference in performance. Training accuracy decreased a little, but validation accuracy rose by 0.5%. This suggests that increasing units does not always lead to better results, possibly due to diminishing returns.
- iv. Add More Layers: Adding more layers did have a positive impact, with the training and validation accuracy improving slightly. This tells us that the deeper models can see more complex patterns but gains are marginal.
- v. Tune Hyperparameters: Hyperparameter tuning using the learning rate, batch size, and epochs generated a significant jump in accuracy, particularly with the training accuracy increasing by 0.7%. This shows us the types of ways in which minor adjustments in training parameters can fine-tune performance.
- vi. Adding Dropout Layer: Adding dropout layers significantly reduced training loss and improved accuracy but made the model less susceptible to overfitting. This was at the expense of a minimal drop in validation accuracy, which suggested the model might have been over-generalized.
- vii. Inserting BatchNormalization: BatchNormalization got the highest training accuracy (96.23%) but introduced some minor rise in validation loss (0.2094), exhibiting overfitting. This suggests that the model is overlearning from the training data, and perhaps there is a requirement for more regularization or optimization.
- viii. RMSprop Optimizer: With the use of the RMSprop optimizer, there were competitive outcomes with better training accuracy (94.84%) and stable validation accuracy (93.66%). The optimizer was effective with the provided architecture with stable training having minimal variations in outcomes.
- ix. Nadam Optimizer: Nadam optimizer acted similar to RMSprop but with a marginally better validation accuracy (93.76%). It also improved training accuracy to 94.78%, suggesting that Nadam might help with marginally better optimization in some cases.

7. Conclusion:

Overall, layer addition, hyperparameter tuning, and usage of optimizers like RMSprop and Nadam assisted with small improvements in training and validation accuracy. BatchNormalization assisted in greatly enhanced training accuracy but loss of validation performance, which may be indicative of an overfitting issue. These findings suggest that an optimal combination of model architecture adjustment and hyperparameter tweaking, along with careful use of regularization, can assist in the creation of the top-performing model for sentiment analysis tasks.